Dịch tài liệu Đồ án Bloxorz

https://gemini.google.com/app/95283fc3cce2c6bb

User prompt: Dịch toàn bộ sang Tiếng Việt cho tôi

Response: Dưới đây là bản dịch toàn bộ nội dung tài liệu sang Tiếng Việt:

CSC14003 Nhập môn Trí tuệ Nhân tạo

Đồ án 1: Bộ giải Bloxorz (Bloxorz Solver)

1. Giới thiệu

Bloxorz[1] là một trò chơi giải đố lăn khối hộp phổ biến, trong đó người chơi điều khiển một khối hộp hình chữ nhật kích thước $1 \times 1 \times 2$ đặt trên một mặt sàn tạo bởi các ô vuông. Khối hộp được lăn quanh bảng, và mục tiêu của trò chơi là làm cho khối hộp rơi thẳng đứng vào một lỗ đích hình vuông.

Thách thức nằm ở chỗ khối hộp không phải là một điểm đơn lẻ: tùy thuộc vào việc nó đang đứng hay nằm, nó sẽ chiếm các ô khác nhau và phản ứng khác nhau với mặt sàn. Lăn khối hộp ra khỏi mép bảng sẽ dẫn đến thua cuộc.

(Hình 1: Ảnh chụp màn hình lối chơi điển hình của Bloxorz)

Trò chơi này đặc biệt phù hợp để thực hành các thuật toán tìm kiếm như BFS, DFS, UCS và A^*. Trò chơi rất dễ tiếp cận: sinh viên có thể nắm bắt luật chơi chỉ trong vài phút và mô hình hóa thành một bài toán tìm kiếm. Trò chơi cũng mang tính quan sát toàn phần (fully observable) và tất định (deterministic): toàn bộ bảng cờ đều có thể nhìn thấy được và mỗi lượt lăn dẫn đến chính xác một trạng thái kế tiếp, không có yếu tố ngẫu nhiên hay đối kháng. Điều thú vị nhất là hướng định vị (orientation) của khối hộp là một phần thiết yếu của trạng thái chứ không chỉ riêng tọa độ của nó, khiến bài toán trở thành một công cụ tuyệt vời để thiết kế các hàm heuristic chấp nhận được (admissible heuristics) và phân tích các phương pháp tìm kiếm có thông tin (informed search).

2. Luật chơi (Gameplay)

Các quy tắc sau đây được phỏng theo phiên bản gốc trên Coolmath Games.

2.1 Cấu tạo và bố cục

Mỗi màn chơi là một lưới hữu hạn các ô vuông. Mỗi ô thuộc một trong các loại sau:

- **Ô sàn (Floor tile):** Ô bình thường, vững chắc mà khối hộp có thể tựa lên.
- **Ô trống / Vực thẳm (Empty cell / void):** Không có ô sàn; nếu bất kỳ phần nào của khối hộp nằm trên ô trống, khối hộp sẽ rơi và nước đi đó không hợp lệ.
- **Lỗ đích (Goal hole):** Ô mục tiêu duy nhất; màn chơi chỉ được giải quyết khi khối hộp rơi thẳng đứng vào lỗ này.

Khối hộp bắt đầu ở trạng thái dựng đứng trên một ô xuất phát được chỉ định.

2.2 Hướng định vị của khối hộp (Block orientation)

Vì khối hộp có kích thước $1 \times 1 \times 2$, nó luôn tồn tại ở một trong ba hướng định vị – đây là thành phần cốt lõi của trạng thái trò chơi:

- **Đứng (Standing):** Khối hộp chiếm một ô đơn lẻ (r, c).
- **Nằm ngang (Lying horizontally):** Khối hộp chiếm hai ô liền kề theo chiều ngang: (r, c) và $(r, c+1)$.
- **Nằm dọc (Lying vertically):** Khối hộp chiếm hai ô liền kề theo chiều dọc: (r, c) và $(r+1, c)$.

2.3 Các nước đi (Moves)

Ở mỗi lượt, người chơi sẽ lăn khối hộp theo một trong bốn hướng: **Lên (Up)**, **Xuống (Down)**, **Trái (Left)**, hoặc **Phải (Right)**. Kết quả phụ thuộc vào hướng hiện tại:

- Khối đứng sẽ bị đổ xuống và nằm trên hai ô theo hướng lăn.
- Khối nằm dọc theo hướng lăn sẽ đứng dựng lên trên một ô đơn.
- Khối nằm vuông góc với hướng lăn sẽ lăn nghiêng và tiếp tục ở trạng thái nằm, dịch chuyển đi một ô.

Một nước đi được coi là **không hợp lệ (illegal)** nếu sau khi lăn, bất kỳ ô nào mà khối hộp chiếm chỗ là ô trống hoặc nằm ngoài bàn cờ. Nói cách khác, toàn bộ khối hộp luôn phải được đỡ trọn vẹn bởi các ô sàn.

2.4 Điều kiện chiến thắng (Victory)

Một màn chơi chiến thắng khi khối hộp rơi vào lỗ đích. Điều này chỉ xảy ra khi khối hộp **dựng đứng** trên ô đích. Khối hộp chỉ nằm đè qua ô đích sẽ không được tính là chiến thắng.

2.5 Các ô đặc biệt (Advanced tiles)

Mỗi màn chơi có thể bao gồm các ô đặc biệt sau. Những ô này bổ sung thêm cơ chế trò chơi bằng cách thay đổi tương tác giữa khối hộp và bàn cờ:

- **Ô dễ vỡ (Fragile tiles - màu cam):** Chỉ có thể đỡ được khối hộp khi trọng lượng của nó được phân bổ đều (khối hộp đang nằm). Nếu khối hộp đứng lên ô dễ vỡ, ô sẽ bị sập và khối hộp sẽ rơi. Khối hộp có thể nằm an toàn đè lên các ô dễ vỡ.
- **Cầu (Bridges):** Là ô có khả năng đi qua được điều khiển bởi một công tắc. Cầu đóng hoạt động như khoảng trống (void), trong khi cầu mở hoạt động như ô sàn bình thường. Mỗi công tắc được liên kết với một hoặc nhiều ô cầu.

Trò chơi có ba loại công tắc:

- **Công tắc nhẹ (Soft switch - hình bát giác / hình tròn):** Được kích hoạt khi bất kỳ phần nào của khối hộp đè lên, bất kể hướng (đứng hay nằm).
- **Công tắc nặng (Heavy switch - hình chữ X):** Chỉ được kích hoạt khi khối hộp đứng thẳng trên đó; nằm lên sẽ không kích hoạt công tắc.
- **Công tắc phân tách (Split switch - hình dấu ngoặc đơn "()"):** Chỉ kích hoạt khi khối hộp đứng trên đó. Nó dịch chuyển tức thời khối hộp đến các vị trí định sẵn và tách nó thành hai khối lập phương nhỏ kích thước $1 \times 1 \times 1$.

Khi được kích hoạt, công tắc tác động lên các cầu liên kết theo một trong hai cách: **chuyển đổi (toggle)** (mỗi lần kích hoạt sẽ đảo trạng thái cầu giữa đóng và mở) hoặc **cố định (permanent)** (cầu được đưa về một trạng thái cố định sau khi kích hoạt).

Phân tách khối hộp (Block splitting): Sau khi bị tách thành hai khối lập phương đơn vị, người chơi điều khiển từng khối một (ví dụ: chuyển đổi quyền điều khiển bằng phím cách - spacebar). Chúng sẽ hợp nhất lại thành khối $1 \times 1 \times 2$ tiêu chuẩn khi được đặt ở hai ô liền kề nhau. Một khối đơn vị có thể kích hoạt công tắc nhẹ nhưng không thể kích hoạt công tắc nặng hay hoàn thành màn chơi.

3. Yêu cầu (Requirements)

Những điểm quan trọng cần lưu ý:

- Mã nguồn phải được viết bằng **Python**.
- Bạn có thể sử dụng các thư viện hỗ trợ (ví dụ: cho GUI hoặc cấu trúc dữ liệu), nhưng **phải tự cài đặt các thuật toán tìm kiếm**.
- Không có ràng buộc về cách tổ chức mã nguồn, nhưng cấu trúc code cần rõ ràng và chuyên nghiệp.

3.1 Tái lập trình trò chơi (20 điểm)

Cài đặt một phiên bản cơ bản và có thể chơi được của Bloxorz. Chương trình **BẮT BUỘC PHẢI CÓ GIAO DIỆN ĐỒ HỌA - GUI** (ứng dụng web hoặc ứng dụng desktop đều được chấp nhận). Phần này gồm hai phần điểm bằng nhau:

- **Cơ chế cốt lõi (10 điểm):** Lối chơi tiêu chuẩn gồm bàn cờ với các ô sàn, ô trống và lỗ đích; khối hộp $1 \times 1 \times 2$ với 3 hướng; 4 hướng lăn với việc chuyển trạng thái hướng chính xác; phát hiện nước đi không hợp lệ (rơi khỏi bàn cờ hoặc rơi vào ô trống); phát hiện chiến thắng (khối đứng vào lỗ đích).
- **Các ô nâng cao (10 điểm):** Ô dễ vỡ, cầu và cả 3 loại công tắc (nhẹ, nặng, phân tách, bao gồm tách/hợp nhất khối và cả hai hành vi của cầu) như mô tả ở Mục 2.5. Các bộ giải cũng phải xử lý đúng các ô này.

Thông tin bổ sung (F.Y.I):

- Chú ý kỹ bảng chuyển đổi hướng: mỗi cặp `(hướng, chiều lăn)` ánh xạ tất định sang `(hướng mới, vị trí mới)`. Cài đặt và viết unit-test bảng này từ sớm sẽ giúp tiết kiệm nhiều thời gian sửa lỗi.
- Đối với công tắc và cầu, trạng thái phải bao gồm trạng thái đóng/mở của từng chiếc cầu, chứ không chỉ vị trí và hướng của khối hộp: các trạng thái có cùng vị trí khối nhưng khác cấu hình cầu là các trạng thái khác nhau.
- Bạn có thể tự thiết kế màn chơi hoặc tái hiện lại các màn cổ điển. Hãy cung cấp định dạng text/JSON đơn giản để nạp màn chơi nhằm tái sử dụng cùng một màn chơi cho tất cả bộ giải.
- GUI cũng phải hiển thị các nút điều khiển cơ bản (ví dụ: Trò chơi mới, Chơi lại, và các nút để gọi các bộ giải bên dưới).

3.2 Bộ giải BFS (10 điểm)

- Cài đặt bộ giải Bloxorz bằng thuật toán Tìm kiếm theo chiều rộng (BFS).
- Bộ giải phải ghi nhận: **Thời gian tìm kiếm (Search Time)**, **Bộ nhớ sử dụng (Memory Usage)**, **Số nút đã duyệt/mở rộng (Expanded Nodes)**, và **Độ dài lời giải (Solution Length)** để phục vụ phân tích trong phần thực nghiệm. Các chỉ số này cũng phải được ghi lại cho từng bộ giải tiếp theo.

3.3 Bộ giải DFS (10 điểm)

- Cài đặt bộ giải Bloxorz bằng thuật toán Tìm kiếm theo chiều sâu (DFS). Bạn có thể sử dụng Tìm kiếm sâu dần (Iterative Deepening Search - IDS) để thay thế nếu muốn; hãy nêu rõ điều này trong báo cáo.
- Xử lý lặp trạng thái để tránh bị kẹt trong chu trình (ví dụ: dùng tập visited hoặc băm - hashing).

3.4 Bộ giải UCS (10 điểm)

- Cài đặt bộ giải Bloxorz bằng thuật toán Tìm kiếm chi phí đồng nhất (UCS).
- Trong báo cáo, định nghĩa rõ ràng và giải thích hợp lý về **hàm chi phí (cost function)** của bạn. Ví dụ: mỗi bước lăn có chi phí là 1 (tối thiểu hóa số bước), hoặc các bước đi khác nhau có chi phí khác nhau (ví dụ: lăn lên ô dễ vỡ thì "đắt" vì rủi ro, trong khi bật công tắc thì "rẻ"), miễn là có lời giải thích hợp lý.
- Lưu ý: nếu mỗi bước đi đều có chi phí là 1, UCS sẽ thoái hóa thành BFS. Việc thiết kế hàm chi phí không đồng nhất (và giải thích trực giác đằng sau nó) chính là điểm tạo nên ý nghĩa cho phần này.

3.5 Bộ giải A^* (10 điểm)

- Cài đặt bộ giải Bloxorz bằng thuật toán Tìm kiếm A^* với một hàm heuristic phù hợp ước lượng khoảng cách từ trạng thái hiện tại đến đích.
- Một số ý tưởng heuristic gợi ý (không bắt buộc):
  - Khoảng cách Manhattan từ khối hộp đến ô đích chia cho 2 (vì một lần lăn có thể di chuyển khối tối đa 2 ô).
  - Thành phần phụ thuộc vào hướng: nếu khối không đứng, nó cần thêm ít nhất một lần lăn nữa để đứng lên ô đích, vì vậy có thể cộng thêm một lượng phạt nhỏ.
  - Khoảng cách "vật thể điểm" nới lỏng: bỏ qua hướng của khối và các ràng buộc ô trống/dễ vỡ, coi khối như một điểm đơn lẻ di chuyển trên lưới kết nối 4 chiều, lấy khoảng cách đường đi ngắn nhất đến đích làm cận dưới.
- Trong báo cáo:
  - Giải thích cách bạn thiết kế hàm heuristic.
  - Thảo luận xem heuristic có chấp nhận được (admissible - không bao giờ đánh giá quá chi phí thực tế còn lại) và/hoặc nhất quán (consistent) hay không, kèm theo lập luận rõ ràng. Cẩn thận: heuristic phải duy trì tính admissible đối với hàm chi phí bạn đã chọn.

3.6 Báo cáo (30 điểm)

Nộp báo cáo định dạng PDF được trình bày chỉn chu, bao gồm:

- **Kế hoạch dự án và phân chia công việc:**

- Ghi rõ trách nhiệm của từng thành viên: công việc được giao và tỷ lệ phần trăm hoàn thành của mỗi người.
- Ví dụ: nếu điểm nhóm là 9.0 và Sinh viên A có tỷ lệ hoàn thành là 90%, thì điểm cá nhân của A là $9.0 \times 90\% = 8.1$.
- Việc đánh giá đóng góp một cách công bằng và chính xác là rất quan trọng.

- **Mô tả thuật toán (10 điểm):**
  - Cung cấp giải thích chi tiết về từng thuật toán tìm kiếm được sử dụng (BFS, DFS/IDS, UCS, A^*).
  - Mô tả cách hình thức hóa bài toán tìm kiếm: biểu diễn trạng thái (bao gồm hướng và cấu hình cầu), các hành động, mô hình chuyển trạng thái, kiểm tra đích (goal test), hàm chi phí, các hàm heuristic và cách tránh duyệt lại trạng thái.
- **Thực nghiệm (20 điểm):**
  - Đánh giá hiệu năng tìm kiếm trên nhiều trường hợp thử nghiệm (sử dụng một tập test cố định để đảm bảo tính công bằng giữa các thuật toán). Các test case có thể được nhóm theo độ khó tăng dần (ví dụ: các trường hợp đơn giản không có ô đặc biệt, có ô dễ vỡ, có công tắc, và kết hợp cả hai), các nhóm này cần được mô tả rõ trong báo cáo.
  - Đo đạc các thông số cho từng thuật toán:
    - Thời gian tìm kiếm.
    - Lượng bộ nhớ sử dụng (đỉnh bộ nhớ - peak memory).
    - Số nút đã mở rộng.
    - Độ dài lời giải (số bước đi).
  - Trình bày kết quả bằng:
    - Bảng tóm tắt các phép đo.
    - Biểu đồ (plots) so sánh giữa các thuật toán.
    - Đưa ra nhận xét, đánh giá sâu sắc về ưu/nhược điểm của từng thuật toán.
- **Tài liệu tham khảo và Phụ lục:**
  - Liệt kê tất cả tài liệu tham khảo (bài báo, sách, bài hướng dẫn trực tuyến, kho lưu trữ GitHub,...).
  - Đính kèm phụ lục, ví dụ: cách bạn sử dụng Generative AI (LLMs) trong quá trình làm đồ án, các prompt đã sử dụng.

3.7 Video (10 điểm)

Tải video demo lên YouTube và đính kèm liên kết công khai (public link) trong báo cáo. Video bao gồm:

- **Giao diện đồ họa (5 điểm):**
  - Trình diễn GUI của chương trình.
  - Thể hiện rõ bố cục chi tiết: bàn cờ với các loại ô khác nhau (sàn, khoảng trống, đích, dễ vỡ, công tắc/cầu), khối hộp ở các hướng khác nhau và các nút điều khiển (Trò chơi mới, Chơi lại, Giải bằng BFS/DFS/UCS/A^*).
- **Trình diễn tính năng (5 điểm):** Trình diễn các tính năng đã cài đặt:
  - Chơi Bloxorz thủ công (lăn khối, kích hoạt công tắc, thắng/thua).
  - Chạy các bộ giải (ví dụ: chọn màn chơi, chạy BFS/UCS/A^*, hiển thị lời giải tìm được, có thể hoạt ảnh hóa hoặc phát lại các bước đi).
- Video cần có phụ đề hoặc thuyết minh rõ ràng (bằng tiếng Anh hoặc tiếng Việt).

4. Quy định nộp bài (Submission)

Vui lòng tuân thủ các hướng dẫn nộp bài sau:

- Mã nguồn và báo cáo phải được đóng gói dưới dạng tệp nén (`.zip`, `.rar`) và đặt tên theo định dạng: `MSSV1_MSSV2_...`
- Nếu tệp nén lớn hơn 20MB, hãy tải lên Google Drive và chia sẻ bằng liên kết. Tuyệt đối không được chỉnh sửa sau thời hạn nộp bài.

Cấu trúc thư mục mẫu:

```
MSSV1_MSSV2_...
├── Source/
│   ├── main.py
│   ├── README.md (hướng dẫn chạy mã nguồn)
│   └── requirements.txt (các thư viện cần cài đặt)
└── Report.pdf (bao gồm URL video demo)
```

5. Lưu ý quan trọng (Notices)

- Đây là bài tập **LÀM THEO NHÓM**.
- Thời gian thực hiện: khoảng 3 tuần.
- Không cấm sử dụng các công cụ AI; tuy nhiên sinh viên nên sử dụng chúng một cách thông minh. Giảng viên hướng dẫn thực hành có quyền phỏng vấn vấn đáp ngẫu nhiên các nhóm để đánh giá mức độ hiểu biết về đồ án.
- Mọi hình thức đạo văn, gian lận hoặc vi phạm quy chế sẽ nhận điểm 0 cho toàn bộ môn học.

Khoa Công nghệ Thông tin - Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM

---

*User prompt: Tôi là leader của một team 5 người. Tôi và Tuấn Anh sẽ làm phần lập trình game, Ân và Vinh sẽ làm BDF solver và DFS solver, Tanh và Khang sẽ làm UCS solver và A*solver, tôi Vinh Khang sẽ làm phần report và Ân (tôi hỗ trợ) sẽ làm phần video. Bạn hãy phân tích cụ thể công việc cần làm ở mỗi phần*

Response: Lưu ý nhỏ là danh sách bạn liệt kê đang có 6 người (Bạn, Tuấn Anh, Ân, Vinh, Tanh, Khang) dù bạn đề cập đây là team 5 người. Tuy nhiên, dựa trên phân công cụ thể bạn đã đưa ra, dưới đây là phân tích chi tiết các đầu việc, yêu cầu kỹ thuật và các điểm cần lưu ý cho từng nhóm dựa trên tài liệu **AI-Project-1.pdf**.

1. Nhóm Lập trình Game (Bạn & Tuấn Anh) Đây là nền tảng cốt lõi của toàn bộ đồ án, các nhóm thuật toán sẽ phụ thuộc vào môi trường game mà hai bạn xây dựng.

- **Công nghệ & Kiến trúc:** Viết bằng Python và bắt buộc phải có Giao diện đồ họa (GUI) dạng web hoặc desktop.
- **Cơ chế cốt lõi (10 điểm):** Cài đặt bàn cờ lưới, xử lý khối hộp $1 \times 1 \times 2$ với 3 trạng thái hướng (đứng, nằm ngang, nằm dọc). Cài đặt 4 thao tác lăn (Lên, Xuống, Trái, Phải), bắt lỗi nước đi không hợp lệ (rơi ra ngoài hoặc vào ô trống), và điều kiện chiến thắng (đứng thẳng vào lỗ đích).
- **Cơ chế nâng cao (10 điểm):** Cài đặt ô dễ vỡ (chỉ chịu được khối nằm), cầu (có thể mở/đóng) và 3 loại công tắc (nhẹ, nặng, phân tách). Chú ý đặc biệt đến cơ chế tách khối thành hai khối $1 \times 1 \times 1$ và hợp nhất lại.
- **Cấu trúc dữ liệu & Tích hợp:** Cần thiết kế một định dạng file (ví dụ: JSON hoặc text) để load các màn chơi chuẩn nhằm sử dụng chung cho các bộ giải thuật toán. Tạo các nút điều khiển trên GUI (New Game, Restart, Giải bằng BFS/DFS/UCS/A*) để các nhóm khác dễ dàng gọi hàm.

2. Nhóm BFS & DFS Solver (Ân & Vinh) Nhóm này phụ trách hai thuật toán tìm kiếm cơ bản, tập trung vào việc quản lý trạng thái hiệu quả.

- **Biểu diễn trạng thái:** Chắc chắn rằng trạng thái (State) các bạn đẩy vào hàng đợi (Queue/Stack) phải bao gồm cả hướng của khối hộp và trạng thái đóng/mở của tất cả các cầu trên bản đồ.
- **Bộ giải BFS (10 điểm):** Cài đặt thuật toán Tìm kiếm theo chiều rộng. Cần thiết lập các biến để đếm và ghi nhận 4 thông số: Thời gian chạy, Đỉnh bộ nhớ sử dụng, Số nút đã duyệt (Expanded Nodes) và Độ dài lời giải.
- **Bộ giải DFS (10 điểm):** Cài đặt Tìm kiếm theo chiều sâu (hoặc Tìm kiếm sâu dần - IDS). Đặc biệt lưu ý phải xử lý triệt để việc lặp trạng thái (dùng cấu trúc Set hoặc Hashing) để chương trình không bị kẹt trong chu trình vô tận.

*3. Nhóm UCS & A Solver (Tanh & Khang)** Phần này đòi hỏi tư duy phân tích toán học và thiết kế hàm đánh giá.

- **Bộ giải UCS (10 điểm):** Tìm kiếm chi phí đồng nhất. **Nhiệm vụ tối quan trọng:** Bạn không thể gán chi phí cho mỗi bước đi bằng 1 (vì như thế UCS sẽ thoái hóa thành BFS). Tanh và Khang phải tự định nghĩa một hàm chi phí riêng (ví dụ: lăn lên ô dễ vỡ rủi ro cao nên chi phí cao, bật công tắc chi phí thấp) và phải giải thích được tính hợp lý của nó.
- *Bộ giải A (10 điểm):** Thiết kế hàm heuristic (ví dụ: khoảng cách Manhattan chia 2, hoặc phạt thêm chi phí khi khối hộp đang nằm). Phải chứng minh được heuristic thiết kế là chấp nhận được (admissible) hoặc nhất quán (consistent) dựa trên hàm chi phí mà phần UCS đã đặt ra.
- **Đo lường:** Tương tự nhóm BFS/DFS, phải trích xuất được 4 thông số hiệu năng để làm báo cáo.

4. Nhóm Báo cáo (Bạn, Vinh, Khang) Báo cáo chiếm tỷ trọng điểm rất lớn (30 điểm) và cần số liệu từ tất cả các nhóm trên.

- **Kế hoạch dự án:** Liệt kê rõ công việc và % hoàn thành của từng thành viên (ảnh hưởng trực tiếp đến điểm cá nhân).
- **Mô tả thuật toán (10 điểm):** Trình bày chi tiết cách hình thức hóa bài toán: cấu trúc không gian trạng thái, định nghĩa hành động, transition model, goal test, và đặc biệt là giải thích hàm heuristic/chi phí của nhóm Tanh, Khang.
- **Thực nghiệm & Phân tích (20 điểm):** Thiết kế một tập các màn chơi (Test cases) chia theo độ khó (cơ bản, có ô dễ vỡ, có công tắc, hỗn hợp). Chạy thử 4 thuật toán trên tập test này, vẽ bảng và biểu đồ so sánh 4 thông số kỹ thuật (Thời gian, Bộ nhớ, Node mở rộng, Số bước). Từ đó rút ra kết luận thuật toán nào ưu việt trong tình huống nào.

5. Nhóm Video (Ân & Bạn hỗ trợ) Video là tài liệu minh chứng cho toàn bộ nỗ lực của team (10 điểm).

- **Yêu cầu kỹ thuật:** Đăng video lên YouTube để ở chế độ công khai/không công khai, dẫn link vào file báo cáo. Video bắt buộc phải có thuyết minh (hoặc phụ đề) tiếng Việt/Anh.
- **Trình diễn GUI (5 điểm):** Quay rõ bàn cờ, các loại ô đặc biệt, trạng thái của khối hộp và các nút bấm điều khiển.
- **Trình diễn Tính năng (5 điểm):** Chơi tay một màn cơ bản (có kích hoạt công tắc, thắng/thua), sau đó bật tính năng giải tự động để biểu diễn cách BFS/DFS/UCS/A* tìm đường và hoạt ảnh khối hộp di chuyển đến đích.

User prompt: tôi chia công việc ra như sau: 1 người làm code game, 4 người mỗi người code thuật toán, vậy làm thế nào để kết hợp 4 code thuật toán vào trong code game ấy được

Response: Việc chia công việc thành 1 người làm Game (đóng vai trò là Môi trường/Test harness) và 4 người làm Thuật toán là một chiến lược rất hiệu quả. Để ráp nối mã nguồn của 5 người một cách mượt mà và không gây xung đột, bạn nên thiết kế hệ thống theo nguyên tắc tách biệt logic (Decoupling) bằng cách sử dụng các lớp (Class) và chuẩn hóa kiểu dữ liệu.

Dưới đây là kiến trúc cụ thể để bạn hướng dẫn team:

1. Thống nhất Cấu trúc dữ liệu (Data Structures)

Để đảm bảo code của 4 người kia có thể giao tiếp được với game của bạn, toàn team cần thống nhất một định dạng dữ liệu chung. Việc sử dụng `dataclasses` kết hợp với tính năng kiểm tra kiểu tĩnh (static typing) sẽ giúp mọi người code chuẩn xác hơn và dễ dàng bắt lỗi.

Bộ giải thuật toán cần trả về chuỗi nước đi và 4 thông số hiệu năng theo yêu cầu của đồ án.

```
from dataclasses import dataclass
from typing import List, Tuple, Any

@dataclass
class State:
    block_positions: List[Tuple[int, int]]  # Tọa độ của khối
    orientation: str                        # 'STANDING', 'LAYING_X', 'LAYING_Y'
    switches_status: dict                   # Trạng thái các cầu/công tắc
    # ... các thông tin khác cần thiết

@dataclass
class SearchResult:
    path: List[str]          # Danh sách nước đi, VD: ['UP', 'RIGHT', 'DOWN']
    search_time: float       # Thời gian tìm kiếm[cite: 7]
    peak_memory: float       # Đỉnh bộ nhớ[cite: 7]
    expanded_nodes: int      # Số nút đã mở rộng[cite: 7]
    solution_length: int     # Số bước đi[cite: 7]
```

2. Định nghĩa Interface chung cho Thuật toán

Người làm game (bạn) sẽ viết một Lớp cơ sở (Base Class). 4 người làm thuật toán bắt buộc phải kế thừa lớp này và chỉ viết code bên trong hàm `solve()`.

```
from abc import ABC, abstractmethod

class BaseSolver(ABC):
    @abstractmethod
    def solve(self, initial_state: State, game_env: Any) -> SearchResult:
        """
        initial_state: Trạng thái bắt đầu của bàn cờ
        game_env: Lớp chứa các hàm hỗ trợ từ Game (VD: hàm sinh trạng thái kế tiếp)
        """
        pass
```

Khi đó, 4 file thuật toán sẽ trông như sau (ví dụ với BFS):

```
# file: solvers/bfs_solver.py
from .base_solver import BaseSolver, SearchResult, State

class BFSSolver(BaseSolver):
    def solve(self, initial_state: State, game_env) -> SearchResult:
        # Code hàng đợi (Queue), tập visited...
        # Gọi game_env.get_successors(current_state) để lấy các bước đi hợp lệ
        return SearchResult(path=['UP', 'RIGHT'], search_time=0.1, ...)
```

3. Tách biệt hoàn toàn Game Logic và Đồ họa

Lưu ý cực kỳ quan trọng: Các thuật toán tìm kiếm phải chạy trên dữ liệu ảo (trong RAM), **tuyệt đối không được đụng đến các hàm vẽ đồ họa** trong lúc đang tìm đường.

- **Nhiệm vụ của bạn (Game Dev):** Cung cấp một hàm tĩnh (ví dụ: `get_successors(state) -> List[Tuple[str, State]]`) để 4 người kia gọi. Hàm này nhận vào một `State`, kiểm tra luật chơi (rơi xuống vực, đứng lên ô dễ vỡ...) và trả về danh sách các nước đi hợp lệ kèm theo trạng thái mới.
- **Nhiệm vụ của Thuật toán:** Chỉ duyệt qua các `State` thông qua hàm `get_successors` cho đến khi gặp ô đích, sau đó gom danh sách hành động lại và `return`.

4. Khớp nối code trên Giao diện đồ họa (Animation)

Khi người dùng nhấn nút "Giải bằng BFS" trên giao diện, quy trình ráp nối sẽ diễn ra như sau:

1. Game lấy trạng thái bàn cờ hiện tại tạo thành đối tượng `State`.
2. Khởi tạo `solver = BFSSolver()` và gọi `result = solver.solve(current_state, game_env)`.
3. Nhận về `result.path` (ví dụ: `['UP', 'RIGHT', 'DOWN']`).
4. **Mô phỏng (Animation):** Thay vì dịch chuyển khối hộp đến đích ngay lập tức, bạn sử dụng bộ đếm thời gian (timer) của vòng lặp sự kiện đồ họa để đọc từng phần tử trong `result.path`, từ từ vẽ chuyển động của khối hộp theo từng bước đi và cập nhật màn hình. Điều này sẽ giúp bạn biểu diễn trực quan được kết quả tìm kiếm.

5. Cấu trúc thư mục tối ưu

Để dễ quản lý trên Git/GitHub, bạn nên tổ chức thư mục như sau để code của ai nằm gọn trong file của người đó:

```
Bloxorz_Project/
├── main.py                 # File chạy chính, khởi tạo GUI và Game loop
├── game/
│   ├── engine.py           # Chứa logic game, hàm get_successors, kiểm tra thắng/thua
│   ├── gui.py              # Xử lý đồ họa, vẽ màn hình, nút bấm
│   └── levels/             # Chứa các file text/JSON cấu hình màn chơi
└── solvers/
    ├── __init__.py
    ├── base_solver.py      # Base Class và Dataclasses
```

```
├── bfs_solver.py       # Code của Ân
├── dfs_solver.py       # Code của Vinh
├── ucs_solver.py       # Code của Tanh
└── astar_solver.py     # Code của Khang
```

Với mô hình này, bạn có thể thiết lập các công cụ kiểm tra (như `mypy --strict` và linter như `ruff`) để đảm bảo toàn bộ mã nguồn của 5 người khi gộp lại sẽ nhất quán, không xảy ra lỗi kiểu dữ liệu hoặc xung đột logic.

User prompt: tôi có nên sử dụng github cho đồ án này không

Response: Chắc chắn là có. Với một nhóm 5 người cùng can thiệp vào một hệ thống mã nguồn, sử dụng GitHub gần như là điều kiện bắt buộc để dự án không rơi vào hỗn loạn.

- **Tránh xung đột mã nguồn (Merge Conflict):** Với cấu trúc thư mục đã chia, mỗi thành viên nên tạo một nhánh (branch) riêng cho phần việc của mình (ví dụ: `feature/game-gui`, `feature/bfs-solver`). Khi một người code xong, họ sẽ tạo Pull Request (PR) để gộp vào nhánh `main`. Việc này giúp nhóm tích hợp 4 bộ giải vào mã nguồn game một cách an toàn mà không sợ ghi đè hay làm hỏng code của nhau.
- **Kiểm soát chất lượng tự động:** Để đảm bảo toàn bộ mã nguồn của 5 người luôn chuẩn mực và khớp nối đúng kiểu dữ liệu, bạn có thể thiết lập GitHub Actions để tự động chạy các công cụ kiểm tra tĩnh như `ruff` và `mypy --strict` mỗi khi có người đẩy (push) code mới lên. Điều này đặc biệt hữu ích khi nhóm sử dụng `dataclasses` làm cấu trúc giao tiếp chung.
- **Minh chứng tiến độ cho Báo cáo:** Đồ án yêu cầu nhóm phải ghi lại trách nhiệm và đánh giá chính xác tỷ lệ hoàn thành phần trăm công việc của từng thành viên để tính điểm cá nhân. Tính năng GitHub Issues (để giao việc) và lịch sử Commit/Contributors sẽ cung cấp dữ liệu minh bạch, khách quan để bạn đưa vào báo cáo, đảm bảo tính công bằng.
- **Đóng gói sản phẩm cuối cùng:** Khi đến hạn chót, bạn có một kho lưu trữ tập trung (central repository) chứa phiên bản code ổn định nhất. Bạn chỉ cần tải toàn bộ mã nguồn từ nhánh `main`, bổ sung thêm file `Report.pdf` (chứa link video) và nén thành tệp `.zip` theo đúng định dạng `MSSV1_MSSV2_...` để nộp.

Bạn dự định sẽ review (đánh giá) code của các thành viên trực tiếp thông qua Pull Request trên GitHub, hay sẽ tổ chức các buổi họp nhóm để cùng nhau chạy thử và debug?

User prompt: hãy hướng dẫn cho tôi cách sử dụng github cho đồ án này

Response: **1. Khởi tạo Repository và Phân quyền (Nhiệm vụ của Leader)**

- Tạo một Repository mới trên GitHub (Private).
- Vào **Settings > Collaborators**, mời Tuấn Anh, Ân, Vinh, Tanh, và Khang.
- Tạo file `.gitignore` chuẩn cho Python (để loại bỏ các thư mục rác như `__pycache__`, môi trường ảo `.venv`).
- Khởi tạo bộ khung dự án (như sơ đồ thư mục ở trên) bao gồm `main.py`, `requirements.txt`, và các định nghĩa `dataclass` (như `State`, `SearchResult`). Push bản gốc này lên nhánh `main`.

2. Chiến lược chia nhánh (Branching) Tuyệt đối không ai được push code trực tiếp vào nhánh `main`. Mỗi thành viên khi làm phần việc của mình phải tạo một nhánh riêng từ `main` với cú pháp rõ ràng:

- Tuấn Anh: `feature/game-gui` hoặc `feature/board-logic`
- Ân: `feature/bfs-solver`
- Vinh: `feature/dfs-solver`
- Tanh: `feature/ucs-solver`
- Khang: `feature/astar-solver`
- Team Báo cáo (Bạn, Vinh, Khang): `docs/latex-report`

3. Quy trình làm việc hàng ngày của các thành viên (Workflow)

1. **Cập nhật:** Trước khi code, chạy lệnh `git pull origin main` để lấy các thay đổi mới nhất (ví dụ: cập nhật cấu trúc `State` từ nhóm Game).
2. **Code & Test:** Viết mã nguồn trên nhánh cá nhân.
3. **Lưu thay đổi (Commit):** Viết thông điệp commit rõ ràng. Ví dụ: `git commit -m "feat: implement cost function for UCS"` hoặc `git commit -m "fix: resolve infinite loop in DFS"`.
4. **Đẩy code (Push) & Tạo Pull Request (PR):** Đẩy nhánh lên GitHub (`git push origin <tên-nhánh>`). Sau đó, lên giao diện GitHub bấm **Create Pull Request** để yêu cầu gộp code vào `main`.

4. Code Review và Tự động hóa (CI/CD) Khi ráp nối code của 5 người, việc lệch kiểu dữ liệu hoặc sai cú pháp rất dễ xảy ra.

- Thiết lập một workflow GitHub Actions đơn giản để mỗi khi có người mở Pull Request, hệ thống sẽ tự động chạy lệnh `mypy --strict` và linter `ruff`.
- Điều này đảm bảo các file thuật toán của Ân, Vinh, Tanh, Khang luôn tuân thủ đúng định dạng dữ liệu đầu vào/đầu ra mà bạn quy định, và mã nguồn toàn dự án được format đồng nhất.
- Chỉ khi CI báo xanh (Pass) và Leader kiểm tra logic ổn thỏa, bạn mới bấm **Merge Pull Request** để đưa code vào `main`.

5. Quản lý Báo cáo và Đóng góp

- Nhóm làm báo cáo có thể đẩy các file mã nguồn LaTeX và hình ảnh biểu đồ vào chung repository trong thư mục `docs/`.
- Tiến độ đóng góp của từng người sẽ được GitHub ghi nhận tự động qua tab **Insights > Contributors**. Bạn hãy dùng biểu đồ commit này làm minh chứng khách quan khi viết mục đánh giá phần trăm hoàn thành công việc của từng thành viên trong báo cáo.

Bạn có muốn tôi viết sẵn file cấu hình GitHub Actions (đuôi `.yml`) để tự động chạy kiểm tra `ruff` và `mypy` cho repo của team không?

User prompt: hàm any là hàm gì

Response: Trong Python, `any` có thể được hiểu theo hai ý nghĩa khác nhau tùy vào việc viết thường hay viết hoa. Dựa vào đoạn code tôi cung cấp ở trên và ngôn ngữ lập trình Python, tôi sẽ giải thích cả hai trường hợp:

1. `typing.Any` (Viết hoa - như trong đoạn code tôi viết ở trên) Đây không phải là một hàm, mà là một **kiểu dữ liệu đặc biệt** (Type Hinting) được import từ thư viện `typing`.

- **Ý nghĩa:** `Any` có nghĩa là "bất kỳ kiểu dữ liệu nào". Khi tôi khai báo hàm `def solve(self, initial_state: State, game_env: Any)`, nó báo cho các công cụ kiểm tra lỗi tĩnh (như `mypy` mà bạn đang dùng) rằng: "Biến `game_env` có thể mang bất kỳ kiểu dữ liệu nào (có thể là một Class phức tạp do team Game tự viết), tạm thời đừng báo lỗi kiểu dữ liệu ở đây".
- **Ứng dụng:** Giúp code linh hoạt hơn trong quá trình các team ráp nối chưa chốt xong 100% cấu trúc của Class Game.

2. Hàm `any()` (Viết thường - Hàm có sẵn của Python) Nếu bạn đang hỏi về hàm thực thi logic, thì Python có sẵn một hàm tên là `any()`.

- **Ý nghĩa:** Hàm này nhận vào một danh sách (list, tuple, tập hợp...) và sẽ trả về `True` nếu có **ít nhất một phần tử** trong đó mang giá trị đúng (`True`). Nếu tất cả đều sai (hoặc danh sách rỗng), nó trả về `False`.
- **Ví dụ:**

```
# Ví dụ 1: Có một phần tử True
danh_sach = [False, False, True, False]
print(any(danh_sach))  # Kết quả: True

# Ví dụ 2: Ứng dụng trong game Bloxorz
# Kiểm tra xem có khối vuông nào đang nằm ngoài bàn cờ không
is_out_of_bounds = [False, False] # Cả 2 nửa khối đều nằm an toàn trên bàn cờ
if any(is_out_of_bounds):
    print("Game Over! Khối hộp đã rơi.")
```

Trong quá trình nhóm viết code xử lý các ô cầu, công tắc hay kiểm tra nước đi hợp lệ cho đồ án, hàm `any()` và người anh em của nó là hàm `all()` (trả về True nếu *tất cả* đều True) sẽ được sử dụng rất thường xuyên để làm code ngắn gọn hơn.

User prompt: trạng thái state chỉ có tọa độ của khối thôi sao, không có tọa độ các ô đặc biệt sao

Response: Câu trả lời ngắn gọn là: **Không**. Bạn không nên lưu tọa độ của *tất cả* các ô đặc biệt vào trong đối tượng `State`. Việc biểu diễn trạng thái (State) chỉ được chứa những gì **có thể thay đổi** sau mỗi nước đi.

Dựa theo yêu cầu của đồ án, bạn cần phân biệt rõ giữa dữ liệu tĩnh (thuộc về Bàn cờ - Board) và dữ liệu động (thuộc về Trạng thái - State):

1. Những dữ liệu BẮT BUỘC phải nằm trong `State` (Dữ liệu Động) Tài liệu đồ án nhấn mạnh rõ ràng rằng trạng thái phải bao gồm các yếu tố thay đổi sau đây:

- **Vị trí và Hướng của khối hộp:** Tọa độ hiện tại và hướng định vị (đứng, nằm ngang, nằm dọc).
- **Trạng thái đóng/mở của các cây cầu (Bridges):** Các trạng thái có cùng vị trí khối hộp nhưng khác cấu hình cầu được tính là các trạng thái khác nhau.
- **Trạng thái khối hộp bị tách (nếu kích hoạt ô phân tách):** Bạn cần lưu tọa độ của cả hai khối lập phương $1 \times 1 \times 1$ riêng biệt và khối nào đang được điều khiển.

2. Những dữ liệu KHÔNG ĐƯỢC lưu trong `State` (Dữ liệu Tĩnh)

- Tọa độ của ô đích, ô sàn, hố sâu (void).
- Tọa độ của các công tắc (nhẹ, nặng, phân tách) và các ô dễ vỡ.

Tại sao lại phải tách biệt như vậy? Trong các thuật toán tìm kiếm (BFS, DFS, UCS, A*), số lượng trạng thái (`State`) được sinh ra và đẩy vào bộ nhớ là cực kỳ lớn (lên đến hàng chục nghìn node). Nếu bạn nhồi nhét cả bản đồ bàn cờ (tọa độ các công tắc, ô đích) vào bên trong mỗi `State`, chương trình của nhóm bạn sẽ bị tràn bộ nhớ (Memory Usage) ngay lập tức. Thông số đỉnh bộ nhớ (Peak Memory) là một tiêu chí bắt buộc phải đo đạc và báo cáo.

Cách tổ chức code tối ưu: Nhóm làm game nên thiết kế một lớp `Board` hoặc `GameEnv` để lưu trữ bản đồ tĩnh, và lớp này chỉ được khởi tạo **một lần duy nhất** khi bắt đầu game. Khi chạy thuật toán, hàm `get_successors(state)` sẽ đối chiếu `State` động với `Board` tĩnh để tính toán ra `State` hợp lệ tiếp theo.

```
@dataclass(frozen=True)  # frozen=True giúp State có thể dùng làm key cho Hash/Set trong thuật toán DFS
class State:
    block_positions: tuple      # VD: ((1, 2), (1, 3))
    orientation: str            # 'STANDING', 'LAYING_X', 'LAYING_Y'
    bridges_status: tuple       # Trạng thái cầu, VD: (True, False, True) - True là mở, False là đóng
```

User prompt: phần orientation: str là gì tôi chưa hiểu

Response: `orientation` trong tiếng Anh có nghĩa là **hướng định vị** hoặc **tư thế**.

Trong trò chơi Bloxorz, khối hộp có kích thước $1 \times 1 \times 2$, do đó chiều không gian không đồng nhất. Tài liệu chỉ rõ rằng hướng của khối hộp là một "phần cốt lõi của trạng thái trò chơi" (core part of the game state) và bắt buộc phải đưa vào biểu diễn trạng thái (state representation).

Cụ thể, `orientation` lưu trữ xem khối hộp đang ở tư thế nào trong 3 tư thế sau:

1. **Đứng (Standing):** Khối hộp dựng đứng và chỉ chiếm 1 ô duy nhất trên bàn cờ.
2. **Nằm ngang (Lying horizontally):** Khối hộp nằm ngả ra và chiếm 2 ô liền kề nhau theo chiều ngang của trục tọa độ.
3. **Nằm dọc (Lying vertically):** Khối hộp nằm ngả ra và chiếm 2 ô liền kề nhau theo chiều dọc của trục tọa độ.

Việc ghi nhận hướng này là bắt buộc vì cùng một vị trí (r, c), nếu khối đang đứng mà bạn lăn sang phải thì nó sẽ thành nằm ngang; nhưng nếu nó đang nằm dọc mà lăn sang phải thì nó vẫn giữ nguyên thế nằm dọc và trượt sang cột bên cạnh.

Về chữ `str` (String): `str` chỉ đơn giản là kiểu dữ liệu dạng chuỗi ký tự. Ý tưởng ở đây là gán cho mỗi tư thế một từ khóa dạng văn bản để code dễ đọc, ví dụ: `orientation = 'STANDING'` hoặc `orientation = 'LYING_X'`.

Tuy nhiên, nếu team đang thiết lập mã nguồn với các tiêu chuẩn tĩnh khắt khe và công cụ kiểm tra kiểu (type checking) nghiêm ngặt để đảm bảo code không bị lỗi vặt, thì dùng `str` có một rủi ro nhỏ là dễ gõ sai chính tả (ví dụ gõ nhầm `'STADING'`). Để

code chất lượng và an toàn hơn khi ráp nối 4 thuật toán lại, nhóm có thể cân nhắc sử dụng `Enum` thay vì `str`:

```
from enum import Enum
from dataclasses import dataclass

class Orientation(Enum):
    STANDING = 1
    LYING_HORIZONTAL = 2
    LYING_VERTICAL = 3

@dataclass(frozen=True)
class State:
    block_positions: tuple
    orientation: Orientation  # Dùng Enum thay vì str
    bridges_status: tuple
```

Với cách này, nếu Ân hay Vinh trong lúc code bộ giải DFS/BFS lỡ gõ sai tên hướng, hệ thống kiểm tra kiểu tĩnh sẽ báo lỗi đỏ ngay lập tức trước cả khi chạy game, giúp nhóm tiết kiệm rất nhiều thời gian debug.

User prompt: bây giờ bạn hãy code cho tôi những class cần thiết, và những khuôn mẫu chung để có thể kết nối code game và 4 phần code thuật toán với nhau

Response: Dưới đây là bộ khung mã nguồn (boilerplate code) được thiết kế bằng lập trình hướng đối tượng (OOP) và tuân thủ chặt chẽ việc kiểm tra kiểu tĩnh (static typing). Cấu trúc này sẽ giúp bạn và Tuấn Anh xây dựng logic game độc lập, đồng thời tạo ra một khuôn mẫu chuẩn xác để Ân, Vinh, Tanh và Khang chỉ việc điền logic thuật toán vào mà không lo xung đột hay lỗi kiểu dữ liệu.

1. File `state.py` (Định nghĩa Dữ liệu chung)

Đây là file cốt lõi nhất. Tất cả 5 người đều phải `import` file này. Việc dùng `frozen=True` giúp đối tượng `State` không thể bị thay đổi sau khi tạo (immutable), nhờ đó các bộ giải có thể dùng nó làm "key" cho tập hợp (Set) hoặc từ điển (Dictionary) để đánh dấu trạng thái đã duyệt.

```
from enum import Enum
from dataclasses import dataclass
from typing import Tuple, List, Dict

class Orientation(Enum):
    STANDING = "STANDING"
    LYING_HORIZONTAL = "LYING_HORIZONTAL"
    LYING_VERTICAL = "LYING_VERTICAL"

@dataclass(frozen=True)
class State:
    # Tọa độ của khối hộp. VD: ((1, 2),) nếu đứng, hoặc ((1, 2), (1, 3)) nếu nằm
    block_positions: Tuple[Tuple[int, int], ...]

    # Hướng định vị của khối
    orientation: Orientation

    # Trạng thái các cầu: tuple chứa các giá trị boolean (True = Mở, False = Đóng)
    bridges_status: Tuple[bool, ...]

    # (Tùy chọn) Vị trí khối bị tách nếu dẫm lên công tắc ()
    # split_positions: Tuple[Tuple[int, int], Tuple[int, int]] | None = None

@dataclass
class SearchResult:
    path: List[str]          # Danh sách các bước đi, VD: ['UP', 'RIGHT', 'DOWN']
    search_time: float       # Thời gian tìm kiếm[cite: 7]
    peak_memory: float       # Đỉnh bộ nhớ sử dụng[cite: 7]
    expanded_nodes: int      # Số nút đã duyệt[cite: 7]
    solution_length: int     # Độ dài lời giải (số bước)[cite: 7]
```

2. File `game_env.py` (Khuôn mẫu Môi trường Game)

File này do bạn và Tuấn Anh đảm nhận. Nó chứa bản đồ tĩnh và cung cấp hàm `get_successors` để các bộ giải gọi.

```
from typing import List, Tuple
from state import State, Orientation

class GameEnv:
    def __init__(self, level_data: dict) -> None:
        """
        Khởi tạo bàn cờ từ dữ liệu màn chơi.
        Lưu trữ các dữ liệu tĩnh như: ô đích, ô dễ vỡ, vị trí công tắc...
        """
        self.grid = level_data.get("grid", [])
        self.goal_pos = level_data.get("goal_pos", (0, 0))
        # Khởi tạo các cấu trúc dữ liệu tĩnh khác...

    def get_initial_state(self) -> State:
        """Trả về trạng thái bắt đầu của màn chơi."""
        # Ví dụ tạm thời:
        return State(
            block_positions=((0, 0),),
            orientation=Orientation.STANDING,
            bridges_status=(False, False) # Giả sử có 2 cầu đang đóng
        )

    def is_goal(self, state: State) -> bool:
        """Kiểm tra xem trạng thái hiện tại đã chiến thắng chưa."""
        return (state.orientation == Orientation.STANDING and
                state.block_positions[0] == self.goal_pos)

    def get_successors(self, state: State) -> List[Tuple[str, State]]:
        """
        Hàm quan trọng nhất dành cho 4 người làm thuật toán.
        Sinh ra các trạng thái hợp lệ tiếp theo từ trạng thái hiện tại.
        """
        successors: List[Tuple[str, State]] = []
        moves = ["UP", "DOWN", "LEFT", "RIGHT"]

        for move in moves:
            # 1. Tính toán vị trí và hướng mới của khối hộp
            # 2. Kiểm tra rớt ra ngoài bàn cờ hoặc rơi xuống ô trống (void)
            # 3. Kiểm tra ô dễ vỡ (nếu đứng lên thì bỏ qua move này)
            # 4. Kích hoạt công tắc & cập nhật bridges_status nếu có

            # Nếu nước đi hợp lệ, tạo State mới và đưa vào danh sách
            # is_valid, new_state = self._apply_move(state, move)
            # if is_valid:
            #     successors.append((move, new_state))
            pass

        return successors
```

3. File `base_solver.py` (Giao diện chung cho Thuật toán)

Để đảm bảo Ân, Vinh, Tanh và Khang tuân thủ đúng đầu vào đầu ra, họ phải kế thừa lớp này.

```
from abc import ABC, abstractmethod
from state import State, SearchResult
from game_env import GameEnv

class BaseSolver(ABC):
    @abstractmethod
    def solve(self, initial_state: State, env: GameEnv) -> SearchResult:
        """
```

```
        Hàm thực thi thuật toán tìm kiếm.
        Yêu cầu trả về đối tượng SearchResult.
        """
        pass
```

4. File `bfs_solver.py` (Ví dụ cách nhóm Thuật toán làm việc)

Đây là cách Ân (hoặc Vinh, Tanh, Khang) sẽ viết code dựa trên bộ khung bạn đã chuẩn bị. Họ hoàn toàn không cần quan tâm đến cách bạn vẽ đồ họa hay bắt lỗi dẫm lên công tắc ra sao, chỉ việc tập trung vào thuật toán.

```
import time
import tracemalloc
from collections import deque
from typing import Set

from state import State, SearchResult
from game_env import GameEnv
from base_solver import BaseSolver

class BFSSolver(BaseSolver):
    def solve(self, initial_state: State, env: GameEnv) -> SearchResult:
        tracemalloc.start()
        start_time = time.time()

        queue = deque([(initial_state, [])]) # Queue chứa tuple (State, path)
        visited: Set[State] = set([initial_state])
        expanded_nodes = 0

        final_path = []

        while queue:
            current_state, path = queue.popleft()
            expanded_nodes += 1

            # Kiểm tra đích[cite: 6]
            if env.is_goal(current_state):
                final_path = path
                break

            # Duyệt các trạng thái kề bằng cách gọi env.get_successors[cite: 6]
            for action, next_state in env.get_successors(current_state):
                if next_state not in visited:
                    visited.add(next_state)
                    queue.append((next_state, path + [action]))

        # Lấy thông số hiệu năng
        current_mem, peak_mem = tracemalloc.get_traced_memory()
        tracemalloc.stop()
        search_time = time.time() - start_time

        return SearchResult(
            path=final_path,
            search_time=search_time,
            peak_memory=peak_mem / (1024 * 1024), # Chuyển sang MB
            expanded_nodes=expanded_nodes,
            solution_length=len(final_path)
        )
```

5. File `main.py` (Nơi khớp nối tất cả lại)

Ở file này, bạn sẽ gắn giao diện đồ họa (ví dụ dùng thư viện Pygame hoặc Tkinter) và gọi các thuật toán.

```
from game_env import GameEnv
from bfs_solver import BFSSolver
# from dfs_solver import DFSSolver
```

```
# from ucs_solver import UCSSolver
# from astar_solver import AStarSolver

def main() -> None:
    # 1. Tải bản đồ
    level_data = {"grid": [...], "goal_pos": (5, 5)} # Lấy từ file cấu hình
    env = GameEnv(level_data)
    initial_state = env.get_initial_state()

    # 2. Khởi tạo GUI (Pygame / Tkinter...)
    # gui = GameGUI(env)
    # gui.draw_board(initial_state)

    print("Bắt đầu giải mã bằng BFS...")
    solver = BFSSolver()
    result = solver.solve(initial_state, env)

    print(f"Đã tìm thấy đường đi: {result.path}")
    print(f"Số bước: {result.solution_length}")
    print(f"Đỉnh bộ nhớ (MB): {result.peak_memory:.4f}")

    # 3. Cập nhật GUI để khối hộp tự động di chuyển theo result.path[cite: 7]
    # for action in result.path:
    #     gui.animate_move(action)

if __name__ == "__main__":
    main()
```

Cấu trúc này tích hợp sẵn các kiểu dữ liệu và module chuẩn của Python (`typing`, `dataclasses`, `enum`), nên nếu bạn thiết lập chạy `mypy --strict` hoặc công cụ linting như `ruff` qua GitHub Actions, chúng sẽ tự động rà soát toàn bộ project và đảm bảo code của 5 người luôn nhất quán với nhau.

User prompt: bạn hãy giải thích cụ thể hơn file game_env.py

Response: File `game_env.py` (Môi trường Trò chơi) chính là **"trái tim"** của toàn bộ hệ thống logic. Trong mô hình AI, nó đóng vai trò là **Trọng tài (Môi trường)**.

Bốn bạn code thuật toán (Ân, Vinh, Tanh, Khang) thực chất là những người bị "bịt mắt". Họ không biết bàn cờ trông như thế nào, họ chỉ cầm trong tay `State` hiện tại và liên tục hỏi bạn (thông qua `game_env.py`): *"Từ đây tôi đi được những bước nào hợp lệ?"*.

Vì vậy, nhiệm vụ của bạn và Tuấn Anh trong file này là xây dựng các bộ quy tắc vật lý của trò chơi. Dưới đây là phân tích giải phẫu chi tiết cấu trúc bên trong `game_env.py` mà hai bạn cần viết:

1. Dữ liệu tĩnh và Khởi tạo (`__init__`)

Trong hàm `__init__`, bạn cần đọc file cấu hình màn chơi (ví dụ: file `.txt` hoặc `.json`) và chuyển nó thành cấu trúc dữ liệu trên RAM. Đây là các dữ liệu **bất di bất dịch** trong suốt quá trình chạy thuật toán.

```
class GameEnv:
    def __init__(self, level_data: dict) -> None:
        # Bàn cờ tĩnh dạng ma trận 2D (0: Vực, 1: Sàn, 2: Dễ vỡ, 3: Lỗ đích...)
        self.grid: List[List[int]] = level_data['grid']
        self.rows = len(self.grid)
        self.cols = len(self.grid[0])

        # Lưu tọa độ đích để kiểm tra chiến thắng nhanh hơn
        self.goal_pos: Tuple[int, int] = level_data['goal_pos']

        # Danh sách cấu hình công tắc.
        # Ví dụ lưu một từ điển ánh xạ: Vị trí công tắc -> Danh sách các vị trí cầu bị ảnh hưởng
        self.switches: dict = level_data['switches']
```

2. Hàm hỗ trợ tính toán hướng lăn (`_apply_move` - Private Method)

Khối hộp có quy luật lăn rất đặc thù. Bạn cần viết một hàm nội bộ chuyên tính toán: *Nếu đang ở tọa độ này, tư thế này, mà lăn hướng này thì tọa độ và tư thế mới sẽ là gì?* (Chưa cần biết có rớt xuống vực hay không, cứ tính ra vị trí lý thuyết đã).

```
    def _apply_move(self, state: State, action: str) -> State:
        # action có thể là: "UP", "DOWN", "LEFT", "RIGHT"
        # Dựa vào state.orientation (STANDING, LYING_X, LYING_Y),
        # tính toán state.block_positions mới và orientation mới.

        # Ví dụ logic: Đang ĐỨNG (1 ô) mà đi qua PHẢI (RIGHT)
        # -> Tư thế mới thành NẰM NGANG (LYING_X)
        # -> Chiếm 2 ô: (r, c+1) và (r, c+2)

        # Trả về một đối tượng State mới
        return new_state_theoretically
```

3. Hàm kiểm tra hợp lệ (`_is_valid_state` - Private Method)

Đây là phần khó nhất và dễ sinh bug nhất. Bạn phải viết logic kiểm tra xem cái `new_state_theoretically` vừa tính ở trên có tồn tại được trên bàn cờ không. Nếu hàm này trả về `False`, khối hộp đã chết.

Các luật cần kiểm tra theo yêu cầu đồ án:

1. **Kiểm tra biên:** Tọa độ của các ô khối hộp đang chiếm có bị âm (`< 0`) hoặc vượt quá kích thước bàn cờ (`>= rows, cols`) không?
2. **Kiểm tra rơi tự do:** Khối hộp có đè lên ô số `0` (vực thẳm / void) không?
3. **Kiểm tra ô dễ vỡ (Fragile):** Nếu khối hộp đang ở tư thế `STANDING` (đứng) VÀ đứng lên đúng ô dễ vỡ → rơi vỡ → Không hợp lệ.
4. **Kiểm tra cầu (Bridges):** Nếu khối hộp đè lên ô cầu, phải tra cứu vào biến `state.bridges_status` xem cây cầu đó hiện tại đang Mở (được đi) hay Đóng (rơi xuống vực).

4. Hàm trọng tâm sinh trạng thái kế tiếp (`get_successors`)

Sau khi đã có 2 hàm hỗ trợ trên, hàm `get_successors` sẽ rất ngắn gọn, sạch sẽ và trở thành API để 4 thuật toán gọi.

```
    def get_successors(self, state: State) -> List[Tuple[str, State]]:
        """ Trả về danh sách [(Hành động, Trạng thái mới), ...] """
        successors = []
        moves = ["UP", "DOWN", "LEFT", "RIGHT"]

        for move in moves:
            # Bước 1: Tính trạng thái lý thuyết
            new_state = self._apply_move(state, move)

            # Bước 2: Kiểm tra luật lệ (rơi vực, vỡ gạch...)
            if not self._is_valid_state(new_state):
                continue # Nếu rơi hoặc sai luật thì bỏ qua nước đi này

            # Bước 3: Kiểm tra công tắc (Switches)
            # Nếu new_state hợp lệ và đang đè lên một nút bấm
            # -> Gọi hàm tự viết _trigger_switch() để cập nhật lại bridges_status
            new_state_with_updated_bridges = self._trigger_switch(new_state)

            # Bước 4: Lưu vào danh sách các bước đi hợp lệ
            successors.append((move, new_state_with_updated_bridges))

        return successors
```

5. Xử lý "Bom nổ chậm": Công tắc phân tách (Split Switch)

Theo yêu cầu mục 2.5 nâng cao (10 điểm), có một loại công tắc cực khó là chẻ khối chữ nhật thành hai khối lập phương nhỏ $1 \times 1 \times 1$.

Để giải quyết phần này, bạn và Tuấn Anh cần thảo luận kỹ cách thiết kế `State`. Khi bị tách, người chơi phải dùng phím `Spacebar` (chuyển đổi) để chọn điều khiển khối A hoặc khối B.

- **Hành động (Action) mới:** Bạn phải bổ sung thêm một hành động là `SWITCH_BLOCK` (đổi điều khiển) bên cạnh `UP, DOWN, LEFT, RIGHT`.
- **Sửa đổi `State`:** Tư thế của khối lúc này không còn là `STANDING` hay `LYING` nữa, mà là một trạng thái đặc biệt như `SPLIT`. Trong `get_successors`, nếu đang ở trạng thái `SPLIT`, lệnh `UP` chỉ làm dịch chuyển tọa độ của cái khối lập phương *đang được trỏ (active)*, còn khối kia giữ nguyên. Cần kiểm tra xem sau khi đi, hai khối có nằm sát cạnh nhau không để hợp nhất chúng lại thành hình chữ nhật ban đầu.

Tóm tắt nhiệm vụ của Bạn và Tuấn Anh:

1. **Thiết kế thuật toán vật lý:** Đảm bảo khối không bao giờ lọt lưới (xuyên tường, đi qua cầu đang đóng...).
2. **Viết Unit Test cho game_env:** Đây là việc sống còn. Trước khi giao hàm này cho Ân, Vinh, Tanh, Khang, bạn nên tự tay chạy thử một đoạn kịch bản ngắn (ví dụ: hardcode 5 nước đi) để xem khối hộp có đổi hướng đúng, cầu có đóng/mở đúng không. Nếu `game_env` của bạn bị sai tọa độ hoặc sai luật, cả 4 thuật toán BFS/DFS/UCS/A* của các bạn kia chạy sẽ cho ra kết quả rác, và cả team sẽ không biết lỗi nằm ở phần đồ họa, phần thuật toán hay phần vật lý game.
